



INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL, I. P.
DELEGAÇÃO REGIONAL DO NORTE
CENTRO DE EMPREGO E FORMAÇÃO PROFISSIONAL DE BRAGA

Technical Report - Ultrasonic Reverse-Parking Sensor

T3 - Specialist Technician in Automation, Robotics, and Industrial Control

Bruna Ferreira Gomes Madeira



Cofinanciado pela
União Europeia

Os Fundos Europeus mais próximos de si.



Part I – Base System Description and Analysis

Ultrasonic Measurement Principle

The HC-SR04 ultrasonic sensor determines distance using the time-of-flight (ToF) measurement principle. A microcontroller-generated trigger pulse of approximately 10 μs is applied to the TRIG pin, initiating the emission of an ultrasonic acoustic burst at a frequency of approximately 40 kHz. This ultrasonic wave propagates through the air, is reflected by an obstacle, and returns to the sensor, where it is detected via the ECHO pin.

The time of flight corresponds to the total elapsed time between the emission of the ultrasonic pulse and the reception of the reflected echo. Since the speed of sound in air at room temperature is approximately 343 m/s, the distance to the reflecting object can be calculated from the measured pulse width.

For practical implementation, the empirical relationship

$$Distance(cm) \approx \frac{pulse\ duration\ (\mu s)}{58}$$

Is commonly used. This approximation accounts for the speed of sound ($\approx 0.0343\text{ cm} / \mu\text{s}$) and the fact that the measured time includes both the outbound and return paths of the acoustic signal. The resulting accuracy is sufficient for short-range sensing applications such as automotive reverse-parking assistance systems.

Importance of Using a Timeout in pulseIn()

If no echo signal is received and no timeout parameter is specified, the Arduino pulseIn() function will block indefinitely while waiting for a signal transition. This blocking behaviour halts program execution, preventing updates to all system outputs, including LEDs, buzzer actuation, LCD refresh, and serial communication.

In real-world systems, such behaviour is unacceptable due to several possible operating conditions, including sensor disconnection or failure, poor acoustic reflection caused by absorbent or angled surfaces, and transient environmental disturbances. Safety-related feedback systems must remain responsive under all conditions.

The use of a timeout ensures non-blocking system operation, allowing the software to detect fault conditions, maintain responsiveness, and present explicit error messages (e.g., “NO ECHO”) rather than displaying invalid or misleading distance information.



LED Bar-Gauge Mapping and Human–Machine Interface Considerations

Mapping measured distance values into multiple LED levels provides continuous and intuitive visual feedback, as opposed to a simple binary on/off indication. This approach allows users to rapidly assess obstacle proximity without the need to interpret numerical data.

An LED bar-gauge offers several usability advantages:

- Faster interpretation compared to numeric displays
- Reduced cognitive load on the user
- Visual similarity to automotive dashboards and industrial control interfaces
- Effective recognition through peripheral vision

From a human–machine interface (HMI) perspective, the bar-gauge representation enhances situational awareness and supports safer and more intuitive user interaction, particularly in time-critical scenarios such as vehicle reversing.

Colour Zones and Distance Thresholds

The use of green, yellow, and red colour zones for distance indication is a well-established engineering practice in safety-related systems. These colours are universally associated with operational status levels and therefore require minimal user training:

- Green indicates safe conditions and normal operation
- Yellow signals caution and the need for increased attention
- Red denotes danger and the requirement for immediate action

This standardized colour coding significantly reduces reaction time and improves usability in time-critical scenarios.

For a small electric vehicle, representative distance thresholds may be defined as follows:

- Green zone: distances greater than 60 cm
- Yellow zone: distances between 60 cm and 20 cm
- Red zone: distances below 20 cm

Critical danger zone: distances below 10 cm, indicated by a nearly continuous acoustic warning

These threshold values must be adapted according to vehicle dimensions, braking characteristics, sensor placement, and expected operating conditions to ensure reliable and meaningful feedback.

Buzzer Interface and Operational Safety

The Arduino tone() function is particularly suitable for driving a passive piezoelectric buzzer, as it generates a square-wave signal at a programmable frequency. This enables dynamic adjustment of both pitch and repetition rate as a function of the measured distance, providing intuitive acoustic feedback.



When an active (self-oscillating) buzzer is used, frequency generation is internal to the device, making external pitch control impossible. In such cases, the tone() function is unnecessary, and only simple digital ON/OFF control is available. Consequently, distance-dependent variations in pitch and cadence cannot be implemented.

For applications requiring informative and graded acoustic warnings, passive buzzers are therefore the preferred solution.

Reliability and Safety Considerations

The reliability of an ultrasonic reverse-parking sensor system may be compromised by several potential failure modes. Common issues include wiring faults or intermittent electrical connections, which can lead to sporadic sensor operation or complete signal loss. Additionally, electrical noise generated by motors, switching power supplies, or other high-frequency electronic components may interfere with accurate signal timing. From a physical perspective, poor acoustic reflection caused by angled, irregular, or sound-absorbing surfaces can result in weak or inconsistent echo detection.

To mitigate these risks and enhance system robustness, a number of practical engineering measures can be implemented. Signal stability can be improved by applying averaging or median filtering techniques to the measured distance values. Electrical reliability may be increased by placing decoupling capacitors close to the sensor's supply pins, thereby reducing susceptibility to supply noise. Mechanical reliability depends on ensuring correct sensor alignment, with the ultrasonic transducer positioned as perpendicular as possible to expected obstacles. Furthermore, the use of waterproof enclosures and proper grounding practices protects the system against environmental influences such as moisture and contamination. Finally, implementing plausibility checks, by enforcing minimum and maximum valid distance limits, helps prevent the display or use of erroneous measurements.

Collectively, these measures significantly improve the robustness, safety, and operational reliability of ultrasonic sensing systems when deployed in practical automotive-style environments.

Base System Code (Part I)

```
#include <Arduino.h>
#include <LiquidCrystal.h>
// ----- PINOS -----
#define TRIG_PIN 6
#define ECHO_PIN 5
#define BUZZ_PIN 3
// LCD: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 10, 9, 8, 7);
// Barra LEDs (8)
const uint8_t ledPins[8] = {A0, A1, A2, A3, A4, A5, 2, 4};
// 0-3 verdes, 4-5 amarelos, 6-7 vermelhos
// ----- PARAMETROS -----
const int DIST_MIN_CM = 10; // vermelho crítico
```



```
const int DIST_MAX_CM = 120;    // acima disto consideramos "longe"
// controlo de "beeps"
unsigned long tBeep = 0;
bool beepState = false;
// ----- FUNCOES -----
long lerDistanciaCm() {
    // Pulso TRIG
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(5);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    // Timeout 30 ms (~5 m)
    unsigned long dur = pulseIn(ECHO_PIN, HIGH, 30000UL);
    if (dur == 0) return -1; // sem eco
    // cm = us / 58 (aprox.)
    long cm = (long)(dur / 58UL);
    return cm;
}

void setBarLevel(uint8_t level) {
    // level 0..8 (quantos LEDs acesos)
    for (uint8_t i = 0; i < 8; i++) {
        digitalWrite(ledPins[i], (i < level) ? HIGH : LOW);
    }
}

uint8_t distanciaParaNivel(long cm) {
    // voímetro: quanto mais perto, mais LEDs
    if (cm < 0) return 0;

    if (cm > 80) return 2;    // muito longe: 2 verdes
    if (cm > 60) return 3;
    if (cm > 45) return 4;    // verdes "cheios"
    if (cm > 30) return 5;    // começa amarelo
    if (cm > 20) return 6;    // amarelo mais forte
    if (cm > 10) return 7;    // entra vermelho
    return 8;                // <10 cm: vermelho máximo
}

void buzzerControl(long cm) {
    if (cm < 0) { // sem eco
        noTone(BUZZ_PIN);
        beepState = false;
        return;
    }

    long d = cm;
    if (d < 1) d = 1;
```



```
if (d > DIST_MAX_CM) d = DIST_MAX_CM;

// frequência (grave -> agudo) ao aproximar
int freq = map((int)d, DIST_MAX_CM, DIST_MIN_CM, 250, 2200);
if (freq < 200) freq = 200;
if (freq > 2500) freq = 2500;

// ritmo do beep (ms) ao aproximar
int period = map((int)d, DIST_MAX_CM, DIST_MIN_CM, 600, 80);
if (period < 60) period = 60;

// zona crítica: <10 cm -> tom contínuo
if (cm <= DIST_MIN_CM) {
    tone(BUZZ_PIN, 2400);
    beepState = true;
    return;
}

// beeping
unsigned long now = millis();
if (now - tBeep >= (unsigned long)period) {
    tBeep = now;
    beepState = !beepState;

    if (beepState) tone(BUZZ_PIN, freq);
    else noTone(BUZZ_PIN);
}
}

void mostrarLCD(long cm, uint8_t nivel) {
    lcd.setCursor(0, 0);
    lcd.print("Dist: ");

    if (cm < 0) {
        lcd.print("---- cm ");
    } else {
        if (cm < 100) lcd.print(" ");
        if (cm < 10)  lcd.print(" ");
        lcd.print(cm);
        lcd.print(" cm ");
    }
}

lcd.setCursor(0, 1);
if (cm < 0) {
    lcd.print("NO ECHO / CHECK ");
} else if (cm <= 10) {
    lcd.print("DANGER <10cm ");
} else if (cm <= 20) {
```



```
    lcd.print("VERY CLOSE      ");
} else if (cm <= 60) {
    lcd.print("APPROACHING      ");
} else {
    lcd.print("CLEAR / SAFE      ");
}
}

void mostrarSerial(long cm, uint8_t nivel) {
    Serial.print("Distance(cm): ");
    Serial.print(cm);
    Serial.print(" | LED level: ");
    Serial.print(nivel);
    Serial.print(" | Status: ");

    if (cm < 0) Serial.println("NO ECHO");
    else if (cm <= 10) Serial.println("DANGER");
    else if (cm <= 20) Serial.println("VERY CLOSE");
    else if (cm <= 60) Serial.println("APPROACHING");
    else Serial.println("SAFE");
}

// ----- SETUP / LOOP -----
void setup() {
    Serial.begin(9600);
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(BUZZ_PIN, OUTPUT);
    for (uint8_t i = 0; i < 8; i++) {
        pinMode(ledPins[i], OUTPUT);
        digitalWrite(ledPins[i], LOW);
    }
    lcd.begin(16, 2);
    lcd.clear();
    lcd.print("EV Parking Sensor");
    lcd.setCursor(0, 1);
    lcd.print("Ultrasonic + LEDs");
    delay(1200);
    lcd.clear();
}

void loop() {
    long cm = lerDistanciaCm();
    uint8_t nivel = distanciaParaNivel(cm);
    setBarLevel(nivel);
    buzzerControl(cm);
    mostrarLCD(cm, nivel);
    mostrarSerial(cm, nivel);
    delay(120);
}
```



}

Base System (Simulation 1)

Simulation Link: [Simulation 1 – Base System on Wokwi](#)

Description:

This simulation implements the base system including:

- Ultrasonic distance measurement
- 8-level LED bar for visual feedback
- Adaptive buzzer
- 16×2 LCD display
- Serial output for monitoring

Part II – System Improvements and Extensions

Rationale for Filtering Ultrasonic Measurements

In practical applications, ultrasonic distance sensors frequently exhibit unstable or “jittery” measurement behaviour due to a combination of physical and electrical factors. The angle of the target surface plays a significant role, as surfaces that are not perpendicular to the sensor may reflect the ultrasonic wave away from the receiver, resulting in weak, delayed, or missing echoes. In addition, soft or porous materials such as fabric, foam, or rubber tend to absorb acoustic energy, further reducing echo strength and measurement reliability.

Measurement instability may also arise from multiple acoustic reflections, where ultrasonic waves bounce between several surfaces before returning to the sensor, leading to inconsistent time-of-flight values. From an electrical standpoint, power supply fluctuations, electromagnetic interference from nearby motors, and digital switching noise can introduce timing errors. Furthermore, inherent limitations in microcontroller timing resolution and interrupt latency contribute to small variations in pulse-width measurements.

For these reasons, filtering techniques are essential to obtain stable and meaningful distance estimates, particularly in systems that provide real-time, user-facing feedback through visual indicators or acoustic warnings.

Moving Average Filtering versus Deadband

Moving average filtering and deadband techniques address different aspects of signal instability and are therefore complementary. A moving average filter reduces random noise by calculating the mean of several consecutive distance measurements, resulting in a smoother and more stable input signal. This approach is effective in minimizing short-term fluctuations caused by measurement noise.



In contrast, a deadband defines a minimum threshold of change, typically on the order of 1 to 2 cm, before the system updates its outputs. Small variations around an accepted stable value are intentionally ignored, preventing unnecessary output changes.

When used together, these methods provide enhanced system stability: the moving average improves input signal quality, while the deadband stabilizes output behaviour. This combined approach effectively prevents LED flicker and buzzer “chatter” while preserving the system’s ability to respond promptly to genuine changes in distance.

Voltage Divider for LDR-Based Light Detection

A voltage divider is an electrical circuit consisting of two resistive elements connected in series between a supply voltage and ground. The voltage measured at the midpoint of the divider depends on the ratio of the two resistances and varies predictably with changes in one of the resistances.

In the context of this project:

- The Light-Dependent Resistor (LDR) exhibits a resistance that decreases as incident light intensity increases.
- The fixed resistor acts as a reference, forming a voltage divider in conjunction with the LDR.
- The voltage at the midpoint of the divider varies continuously according to the ambient light level.

An analogue input is necessary to measure this voltage because light intensity is a continuous variable, rather than a binary on/off state. The Arduino’s Analog-to-Digital Converter (ADC) converts the measured voltage into a numerical value, which can then be compared to a calibrated threshold to determine day or night conditions. This information is subsequently used to control the reverse-lamp logic.

Output Driving Considerations for LEDs

When driving a white LED from a microcontroller digital output, a series resistor is required to:

- Limit the current to a safe operating range, typically 10–20 mA
- Protect both the LED and the Arduino output pin from overcurrent
- Ensure predictable and consistent brightness

If a high-power lamp is employed, direct connection to a microcontroller pin is unsafe due to the following reasons:

- The current required exceeds the maximum output capability of the microcontroller
- Excessive current may damage the microcontroller or the lamp
- A switching device, such as a transistor, MOSFET, or relay, is required to control the lamp
- An external power supply and protective components (e.g., flyback diodes or fuses) must be used to ensure safe operation



Implementing these measures guarantees safe, reliable, and predictable operation for both low-power indicator LEDs and higher-power automotive-style lighting applications.

Enhanced System Code (Part II)

```
//PARTE 2 rev 2
```

```
#include <Arduino.h>
#include <LiquidCrystal.h>
// ----- PINOS -----
// Ultrassons
#define TRIG_PIN 6
#define ECHO_PIN 5
// Buzzer
#define BUZZ_PIN 3
// LED branco (marcha-atrás)
#define REVERSE_LED_PIN 2
// LDR (divisor)
#define LDR_PIN A0
// LCD: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 10, 9, 8, 7);

// Barra LEDs (6)
const uint8_t barPins[6] = {A1, A2, A3, A4, A5, 4};
// 0-1 verdes, 2-3 amarelos, 4-5 vermelhos
// ----- PARAMETROS -----
const int DIST_MIN_CM = 10; // vermelho crítico
const int DIST_MAX_CM = 120; // acima disto consideramos "longe"
// Média móvel e zona morta
const uint8_t AVG_N = 5; // 3..5 recomendado
const int DEADBAND_CM = 2; // 1..2 recomendado
const int READ_GAP_MS = 25; // intervalo entre leituras para a média
// LDR (ajustar com base em leitura real)
int LDR_THRESHOLD = 750; // exemplo: abaixo disto = noite
// controlo de "beeps"
unsigned long tBeep = 0;
bool beepState = false;
// valor filtrado/estável
long cmStable = -1;
long cmLastShown = -999;
// ----- FUNCOES -----
long lerDistanciaCmRaw() {
    digitalWrite(TRIG_PIN, LOW);

    delayMicroseconds(5);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
```



```
digitalWrite(TRIG_PIN, LOW);
unsigned long dur = pulseIn(ECHO_PIN, HIGH, 30000UL);
if (dur == 0) return -1;
long cm = (long)(dur / 58UL);
if (cm < 0 || cm > 500) return -1;
return cm;
}

long lerDistanciaCmMedia() {
    long soma = 0;
    uint8_t validas = 0;
    for (uint8_t i = 0; i < AVG_N; i++) {
        long cm = lerDistanciaCmRaw();
        if (cm > 0 && cm < 500) {
            soma += cm;
            validas++;
        }
        delay(READ_GAP_MS);
    }
    if (validas == 0) return -1;
    return (soma / validas);
}

void setBarLevel(uint8_t level) {
    // level 0..6 (quantos LEDs acesos)
    for (uint8_t i = 0; i < 6; i++) {
        digitalWrite(barPins[i], (i < level) ? HIGH : LOW);
    }
}

uint8_t distanciaParaNivel6(long cm) {
    if (cm < 0) return 0;
    // Quanto mais perto, mais LEDs (6 níveis)
    if (cm > 80) return 1; // longe: 1 verde
    if (cm > 60) return 2; // 2 verdes
    if (cm > 40) return 3; // entra amarelo
    if (cm > 20) return 4; // 2 amarelos
    if (cm > 10) return 5; // entra vermelho
    return 6; // <=10 cm: vermelho máximo
}

void buzzerControl(long cm) {
    if (cm < 0) {
        noTone(BUZZ_PIN);
        beepState = false;
        return;
    }
    long d = cm;
    if (d < 1) d = 1;

    if (d > DIST_MAX_CM) d = DIST_MAX_CM;
```



```
// grave -> agudo
int freq = map((int)d, DIST_MAX_CM, DIST_MIN_CM, 250, 2200);
if (freq < 200) freq = 200;
if (freq > 2500) freq = 2500;
// lento -> rápido
int period = map((int)d, DIST_MAX_CM, DIST_MIN_CM, 650, 80);
if (period < 60) period = 60;
// crítico: contínuo
if (cm <= DIST_MIN_CM) {
  tone(BUZZ_PIN, 2400);
  beepState = true;
  return;
}
unsigned long now = millis();
if (now - tBeep >= (unsigned long)period) {
  tBeep = now;
  beepState = !beepState;
  if (beepState) tone(BUZZ_PIN, freq);
  else noTone(BUZZ_PIN);
}
}
bool isNight(int ldrVal) {

  // regra típica: valor mais baixo = mais escuro
  return (ldrVal < LDR_THRESHOLD);
}

void mostrarLCD(long cm, int ldrVal, bool noite) {
  lcd.setCursor(0, 0);
  lcd.print("Dist:");
  if (cm < 0) {
    lcd.print(" ---cm ");
  } else {
    if (cm < 100) lcd.print(" ");
    if (cm < 10) lcd.print(" ");
    lcd.print(cm);
    lcd.print("cm ");
  }
  lcd.print(noite ? "N" : "D"); // Night/Day
  lcd.setCursor(0, 1);
  if (cm < 0) {
    lcd.print("NO ECHO / CHECK ");
  } else if (cm <= 10) {
    lcd.print("DANGER <10cm ");
  } else if (cm <= 20) {
    lcd.print("VERY CLOSE ");
  } else if (cm <= 60) {
    lcd.print("APPROACHING ");
  } else {
```



```
lcd.print("CLEAR / SAFE ");
}
}
void mostrarSerial(long cm, uint8_t nivel, int ldrVal, bool noite) {
  Serial.print("Distance(cm): ");
  Serial.print(cm);
  Serial.print(" | Bar: ");
  Serial.print(nivel);
  Serial.print(" | LDR: ");
  Serial.print(ldrVal);
  Serial.print(" | ");
  Serial.print(noite ? "NIGHT" : "DAY");
  Serial.print(" | ReverseLED: ");
  Serial.println(noite ? "ON" : "OFF");
}
// ----- SETUP / LOOP -----
void setup() {
  Serial.begin(9600);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZ_PIN, OUTPUT);
  pinMode(REVERSE_LED_PIN, OUTPUT);
  digitalWrite(REVERSE_LED_PIN, LOW);

  // barra LEDs
  for (uint8_t i = 0; i < 6; i++) {
    pinMode(barPins[i], OUTPUT);
    digitalWrite(barPins[i], LOW);
  }
  // LDR é entrada analógica (não precisa pinMode)
  lcd.begin(16, 2);
  lcd.clear();
  lcd.print("EV Parking Sensor");
  lcd.setCursor(0, 1);
  lcd.print("6LED + LDR");
  delay(1200);
  lcd.clear();
  // primeira leitura estável
  cmStable = lerDistanciaCmMedia();
  cmLastShown = -999;
}
void loop() {
  // 1) Leitura média do ultrassons
  long cmAvg = lerDistanciaCmMedia();
  // 2) Zona morta (deadband)
  bool aceitar = false;
```



```
if (cmStable < 0 && cmAvg >= 0) {  
  aceitar = true;  
} else if (cmAvg < 0) {  
  aceitar = true; // mostrar falha de eco  
} else {  
  long diff = cmAvg - cmStable;  
  if (diff < 0) diff = -diff;  
  if (diff >= DEADBAND_CM) aceitar = true;  
}  
if (aceitar) cmStable = cmAvg;  
// 3) LDR (noite/dia)  
int ldrVal = analogRead(LDR_PIN);  
bool noite = isNight(ldrVal);  
// 4) LED branco (lanterna marcha-atrás) liga se noite  
digitalWrite(REVERSE_LED_PIN, noite ? HIGH : LOW);  
// 5) Saídas do sensor de distância  
uint8_t nivel = distanciaParaNivel6(cmStable);  
setBarLevel(nivel);  
buzzerControl(cmStable);  
// 6) LCD: só reescrever se mudou a distância ou mudou noite/dia  
static bool lastNoite = false;  
bool mudou = (cmStable != cmLastShown) || (noite != lastNoite);  
if (mudou) {  
  
  mostrarLCD(cmStable, ldrVal, noite);  
  cmLastShown = cmStable;  
  lastNoite = noite;  
}  
// 7) Serial (podes pôr só quando muda, mas aqui é sempre)  
mostrarSerial(cmStable, nivel, ldrVal, noite);  
delay(120);  
}
```

Enhanced System (Simulation 2)

Simulation Link: [Simulation 2 – Enhanced System on Wokwi](#)

Description:

This simulation extends the base system with:

- Moving average filtering for stable distance readings
- Deadband logic to prevent LED flicker and buzzer chatter
- LDR-based night/day detection
- White reverse LED activation at night